

Atividade Prática

Projeto de Deep Learning para Demandas Reais

Toxicidade e Inclinação Política

em Dois Registros da Linguagem Brasileira

Alunos: Cleber Henrique Lacerda Duarte, Pedro Lucas Alves de Assis Cardoso, Guilherme Reis de Souza Silva Ferreira e Rafael Perreira de Souza, James Dean Vieira dos Santos, Pedro Augusto Ribeiro Fonseca Guedes.

Análise computacional comparada de toxicidade no discurso cidadão e de sinal ideológico no discurso institucional, com teste de transferência entre registros.

Campo	Conteúdo
Tema	Processamento de Linguagem Natural / Análise de Comportamento
Técnica principal	Deep Learning — Transformers (BERT) com fine-tuning
Bases de dados	ToLD-Br (toxicidade) + Discursos da Câmara dos Deputados (ideologia)
Integrantes	_____
Disciplina / Professor	Robson — _____
Data	25/06/2026

Sumário

(Para atualizar a numeração no Word: clique no sumário → botão direito → Atualizar campo.)

Resumo

Este projeto desenvolve uma solução de Deep Learning, em português brasileiro, para duas tarefas de classificação de texto que representam faces distintas da linguagem política: (1) detecção de toxicidade no discurso cidadão e informal (tweets) e (2) classificação de inclinação ideológica no discurso institucional e formal (discursos parlamentares). Mais do que dois classificadores, o trabalho investiga uma pergunta de pesquisa: representações de linguagem aprendidas em um registro transferem para o outro, ou um modelo compartilhado sofre transferência negativa?

Como evidência concreta, treinamos oito modelos: na toxicidade, baseline LogReg (0,70), MLP (0,67) e BERT (0,77); na inclinação política, após corrigir um vazamento de rótulo, baseline (0,77), MLP (0,78), BERTimbau truncado (0,73) e com janelamento (0,77); e um experimento multi-task que revelou transferência ASSIMÉTRICA entre os registros. O projeto entrega o pipeline reprodutível completo, uma página web interativa de demonstração, e a documentação técnica das sete etapas exigidas.

1. Definição do Problema

1.1 Contexto do problema

A esfera pública brasileira se expressa em registros linguísticos muito diferentes. De um lado, o cidadão comum em redes sociais produz texto curto e informal, com gírias, ironia e, com frequência, toxicidade. De outro, o agente institucional — o parlamentar — produz texto longo, formal e protocolar, mas carregado de sinal ideológico. Compreender automaticamente esses sinais de comportamento político é uma demanda real de moderação de plataformas, observatórios de discurso de ódio, análise de polarização e transparência legislativa.

1.2 Impacto do problema

O volume de texto político e tóxico produzido diariamente é humanamente irrevisável. A toxicidade não moderada amplifica discurso de ódio e afasta usuários; a opacidade do discurso institucional dificulta o escrutínio democrático. Ferramentas automáticas de apoio têm, portanto, impacto direto na qualidade do debate público e na segurança das plataformas.

1.3 Quem é afetado

São afetados: usuários de redes sociais expostos a conteúdo tóxico; equipes de moderação e gestores de comunidade que precisam triar grandes volumes; pesquisadores de ciência política e jornalistas de dados que analisam discurso; e a sociedade, na medida em que polarização e discurso de ódio degradam o debate democrático.

1.4 Por que o problema é relevante

Português brasileiro é uma língua de baixa cobertura em recursos de PLN para postura política, e a maioria dos modelos de toxicidade e de análise política é construída e avaliada dentro de um único registro, assumindo silenciosamente que o que funciona em um vale para o outro. Esse ponto cego — a suposição de transferência entre registros — raramente é testado e é justamente o que este projeto investiga, o que torna o trabalho relevante tanto na aplicação quanto na contribuição metodológica.

1.5 Limitações da solução proposta

- A toxicidade é tratada de forma binária (tóxico / não-tóxico); subtipos raros (racismo, xenofobia) são subrepresentados na base e não são alvo principal.
- A inclinação ideológica usa supervisão distante: o rótulo vem do partido do deputado, não de anotação por fala — há ruído (um parlamentar pode discursar contra a linha do próprio partido).
- Os dois registros diferem de domínio; a solução não pretende que um modelo treinado em um registro funcione no outro — ao contrário, mede essa falha.
- O sistema é ferramenta de apoio, não decisão final: pressupõe humano no circuito.

1.6 Justificativa do uso de Deep Learning

Três propriedades do problema justificam Deep Learning frente a abordagens rasas. A toxicidade depende de contexto, ironia e construções não literais ("que pessoa maravilhosa,

hein"), que modelos lineares de superfície capturam mal. A inclinação ideológica em discurso formal é um sinal difuso e de longo alcance — não está numa palavra-chave, mas no enquadramento ao longo de parágrafos. E a própria pergunta de transferência só é formulável em termos de representações compartilhadas, conceito nativo de Deep Learning (encoders pré-treinados, multi-task learning). Nossa evidência empírica reforça isso: o baseline raso de contagem de palavras erra sistematicamente textos irônicos e de ataque (ver Seção 6), enquanto modelos contextuais (BERT) são projetados para resolvê-los.

2. Objetivo da Solução

2.1 O que o sistema pretende fazer

Desenvolver, documentar e demonstrar uma solução de Deep Learning capaz de (i) classificar toxicidade em discurso cidadão, (ii) classificar inclinação ideológica em discurso institucional e (iii) medir empiricamente se há transferência de representações entre esses dois registros.

2.2 Qual resultado espera produzir

- Dois classificadores binários robustos, um por tarefa, com codificador adequado a cada registro.
- Uma comparação metodologicamente limpa entre baselines independentes e uma arquitetura multi-tarefa de encoder compartilhado.
- Uma conclusão sustentada sobre a hipótese de transferência entre registros — inclusive se for negativa.

2.3 Qual parte do problema será solucionada

O projeto resolve a etapa de classificação automática (triagem de toxicidade e mapeamento de inclinação ideológica em texto), que é a parte da demanda em que Deep Learning contribui de forma mais direta. Não pretende substituir o julgamento humano de moderação nem inferir convicções de indivíduos privados.

2.4 Qual ganho esperado a solução oferece

Ganho de escala (triar volumes humanamente irrevisáveis), ganho analítico (mapear tendências ideológicas em grandes corpora) e ganho metodológico (evidência sobre a (não) transferência entre registros, que orienta projetos futuros e evita reuso indevido de modelos).

Hipóteses de trabalho

- **H1** — codificadores monolíngues em português superam alternativas multilíngues genéricas.
- **H2** — a transferência entre registros é fraca ou negativa: o encoder compartilhado não supera (e pode piorar) os baselines especializados. Confirmar ou refutar H2 é o resultado central.
- **H3** — um modelo treinado em um registro tem desempenho ruim aplicado diretamente ao outro.

3. Base de Dados

O projeto usa duas bases reais, escolhidas com dois critérios inegociáveis para reprodutibilidade: o texto precisa estar disponível de forma aberta (não apenas como identificadores) e a licença precisa permitir reuso em pesquisa.

Frente	Base	Origem	Licença	Rótulo
Toxicidade	ToLD-Br	Twitter PT-BR	CC BY-SA 4.0	Binário: tóxico / não-tóxico
Inclinação	Discursos da Câmara	API de dados abertos da Câmara	Dados públicos	Binário: esquerda / direita

3.1 Base 1 — Toxicidade (ToLD-Br)

Origem, tipo e quantidade

ToLD-Br (Leite et al., 2020) é uma coleção de tweets em português brasileiro anotados quanto a toxicidade. Tipo de dado: texto curto de rede social. O arquivo ToLD-BR.csv contém 21.000 tweets; após deduplicação de textos idênticos, restam 20.813 exemplos efetivos. Cada tweet traz seis colunas de categoria (homophobia, obscene, insult, racism, misogyny, xenophobia), cada uma com a contagem de votos (0 a 3) dos três anotadores.

Estratégia de coleta

A base é pública e versionada; foi obtida diretamente do repositório oficial no GitHub (clonagem do repositório), sem necessidade de reidratação — o que garante reprodutibilidade imediata.

Estratégia de tratamento e pré-processamento

As menções a usuários já vêm anonimizadas como @user. O rótulo binário é derivado pela regra de maioria: um tweet é considerado tóxico se alguma categoria recebeu pelo menos 2 dos 3 votos. Aplica-se deduplicação de textos idênticos e vetorização TF-IDF (uni e bigramas, min_df=3, sublinear_tf) para o baseline; para os modelos profundos, tokenização do próprio modelo pré-treinado.

Balanceamento

A base é naturalmente desbalanceada: 19,4% de exemplos tóxicos no limiar de 2 votos. Em vez de reamostrar (o que descartaria dados ou criaria duplicatas sintéticas), trata-se o desbalanceamento na função de perda, com pesos de classe inversamente proporcionais à frequência (class_weight = balanced).

Distribuição medida

Recorte	Positivos	Proporção
Binário, ≥ 1 voto	9.255	44,1%
Binário, ≥ 2 votos (usado)	4.063	19,3%
obscene (≥ 2)	2.403	11,4%
insult (≥ 2)	1.869	8,9%
homophobia / misogyny / xenophobia / racism (≥ 2)	176 / 133 / 42 / 33	< 1% cada

A leitura desta tabela motivou a decisão de granularidade: quatro das seis categorias têm menos de 200 exemplos, inviabilizando um multi-rótulo confiável. Por isso adota-se o rótulo binário, estatisticamente saudável.

Separação treino / validação / teste

Divisão estratificada por classe em 70% / 15% / 15%, preservando a proporção de positivos em cada conjunto. No experimento executado: 14.569 exemplos de treino, 3.122 de validação e 3.122 de teste.

Tratamento de ruído e inconsistências

O ruído de anotação é mitigado pela regra de maioria (≥ 2 votos), que descarta marcações isoladas de um único anotador. Textos duplicados são removidos. Para comparabilidade com a literatura, há também o split oficial 3annotator distribuído pelos autores.

Limitações da base

Reflete o Twitter de um período específico (2017–2020); subtipos raros de toxicidade são subrepresentados; e há o viés conhecido de bases de toxicidade de marcar em excesso dialetos minoritários e termos reapropriados.

3.2 Base 2 — Inclinação política (Discursos da Câmara)

Por que não o UStanceBR

A proposta inicial previa o UStanceBR (postura político em Twitter). Ao inspecionar o pacote oficial, constatou-se que todo o corpus é distribuído apenas como identificadores de tweets (Tweet_ID; Polarity), sem o texto. Reidratar via API do Twitter/X é inviável de forma reprodutível em 2026 (API paga e restrita; tweets de 2018–2020 amplamente removidos). Por isso o UStanceBR foi descartado como fonte primária — decisão documentada na Seção 9.

Origem, tipo, quantidade e coleta

Optou-se pelos discursos da Câmara dos Deputados, obtidos pela API de dados abertos (dadosabertos.camara.leg.br). Tipo de dado: transcrições de discurso (texto formal longo). A coleta é programática, encadeando o endpoint de deputados (que fornece id e siglaPartido) com o endpoint de discursos (campo transcricao). O volume disponível é de dezenas de milhares de discursos por legislatura. Um script de coleta parametrizável (coletar_discursos_camara.py) acompanha o projeto.

Estratégia de tratamento, rotulagem e ruído

O rótulo de ideologia (esquerda / direita) é atribuído pelo partido do deputado, mapeado para o eixo esquerda–direita com base na classificação de especialistas de Bolognesi et al. (2023). Discursos muito curtos são descartados (parâmetro min-palavras) para reduzir ruído de falas protocolares. O ruído da supervisão distante é assumido e analisado por partido na avaliação.

Balanceamento, separação e vazamento

Para o eixo binário, a faixa de centro é descartada, deixando classes relativamente equilibradas. A separação treino/teste é AGRUPADA POR DEPUTADO (GroupShuffleSplit): nenhum deputado aparece em treino e teste ao mesmo tempo. Isso evita que o modelo

aprenda o estilo idiossincrático do autor e "vaze" sua identidade — medindo qual ideologia, não quem fala.

Observação de ambiente

A coleta deve ser executada na máquina do usuário, pois o ambiente de sandbox deste desenvolvimento não tem acesso de rede ao domínio gov.br. Por essa razão, a evidência quantitativa apresentada na Seção 6 refere-se à frente de toxicidade (base disponível localmente); a frente política dispõe do pipeline completo, pronto para execução após a coleta.

4. Modelagem em Deep Learning

4.1 Estratégia geral

A modelagem segue três blocos em ordem crescente de ambição, para que cada conclusão se apoie na anterior: (1) um baseline raso (TF-IDF + Regressão Logística) que quantifica o quanto o Deep Learning agrega; (2) baselines profundos independentes (um Transformer por tarefa) — o modelo principal; e (3) uma arquitetura multi-tarefa de encoder compartilhado, que testa diretamente a transferência entre registros.

4.2 Tipo de rede e justificativa

A arquitetura escolhida é o Transformer pré-treinado (BERT), com fine-tuning. Justificativa técnica: frente a CNNs (boas em n-gramas locais, fracas em ironia e contexto não local) e LSTMs (que degradam em dependências muito longas, como o sinal ideológico difuso do discurso), a atenção global do Transformer captura dependências de longo alcance, e o pré-treino em grandes corpora em português injeta conhecimento linguístico que nossas bases (dezenas de milhares de exemplos) não ensinariam do zero.

Codificadores casados ao registro

A decisão central — e a que fundamenta os dois modelos separados — é casar o codificador ao registro de cada tarefa:

Tarefa	Codificador	Pré-treino	Por quê
Toxicidade	BERTabaporu (base, uncased)	~237 mi de tweets em PT	Mesmo domínio (rede social); aprende gíria e emoji
Inclinação	BERTimbau (base, cased)	BrWaC (texto formal)	Domínio formal; cased preserva nomes e siglas
Multi-task	Encoder único + 2 cabeças	—	Testa a transferência entre registros (H2)

Como não existe um único encoder ótimo para os dois registros, nasce a hipótese de transferência negativa (H2). Como controle de H1, cada tarefa é também treinada com mBERT multilíngue para comparação.

4.3 Função de ativação

Internamente, os Transformers usam ativação GELU nas camadas feed-forward. Na camada de saída de classificação aplica-se softmax (duas classes), que converte logits em probabilidades. No baseline raso, a Regressão Logística usa a função logística (sigmoide).

4.4 Função de perda

Entropia cruzada com pesos de classe (CrossEntropyLoss ponderada), inversamente proporcionais à frequência, para que a classe minoritária (tóxico) não seja ignorada. Registra-se como alternativa a focal loss, caso o F1 da classe positiva fique aquém.

4.5 Otimizador

AdamW (Adam com weight decay desacoplado), padrão para fine-tuning de Transformers, com warmup de 10% dos passos e weight decay de 0,01. O baseline raso usa o solver lbfgs da Regressão Logística.

4.6 Métricas de avaliação

Métrica primária: macro-F1 (robusta ao desbalanceamento). Secundárias: precisão, revocação e F1 por classe; acurácia (apenas contextual); ROC-AUC; e matriz de confusão. Métricas de regressão (MAE, RMSE) não se aplicam, por se tratar de classificação — opção registrada explicitamente.

4.7 Quantidade de épocas e batch size

Hiperparâmetro	Toxicidade	Inclinação	Justificativa
Épocas	3–4 (early stopping)	3 (early stopping)	BERT converge rápido; mais épocas → overfitting
Batch size	32	8	Discursos são longos → batch menor por memória
max_len	128	512 (+ janelamento)	Tweets curtos; discursos exigem contexto longo
Taxa de aprendizado	2e-5	2e-5	Faixa segura para não destruir o pré-treino

4.8 Estratégias contra overfitting

Early stopping monitorando a macro-F1 de validação; weight decay (0,01); dropout (0,1); número baixo de épocas; e, opcionalmente, layer-wise learning rate decay. Para discursos longos, janelamento em janelas de 512 tokens com sobreposição e agregação por média das probabilidades, preservando o sinal difuso sem trancar.

4.9 Como o modelo é treinado

Cada tarefa é treinada de forma independente por fine-tuning completo do encoder (todas as camadas), com a cabeça de classificação sobre o token [CLS]. O multi-task usa hard parameter sharing: como as bases são disjuntas (nenhum texto tem os dois rótulos), o treino alterna batches em round-robin — cada batch atualiza o encoder e apenas a cabeça correspondente, com mascaramento da perda da outra tarefa. Tudo com sementes fixas e versões de biblioteca registradas.

5. Avaliação dos Resultados

Esta seção apresenta resultados quantitativos REAIS do baseline executado e o protocolo de avaliação dos modelos profundos. A métrica primária é a macro-F1; métricas de regressão (MAE/RMSE) não se aplicam por ser tarefa de classificação.

5.1 Resultado quantitativo — baseline executado (toxicidade)

Foi treinado um baseline TF-IDF + Regressão Logística sobre o ToLD-Br (20.813 exemplos após deduplicação; 14.569 treino / 3.122 validação / 3.122 teste). Resultados no conjunto de teste:

Métrica	Valor
Acurácia	79,7%
macro-F1 (primária)	69,7%
ROC-AUC	77,5%
Precisão — não-tóxico / tóxico	89,2% / 48,0%
Revocação — não-tóxico / tóxico	85,2% / 57,1%
F1 — não-tóxico / tóxico	87,1% / 52,1%

5.2 Matriz de confusão

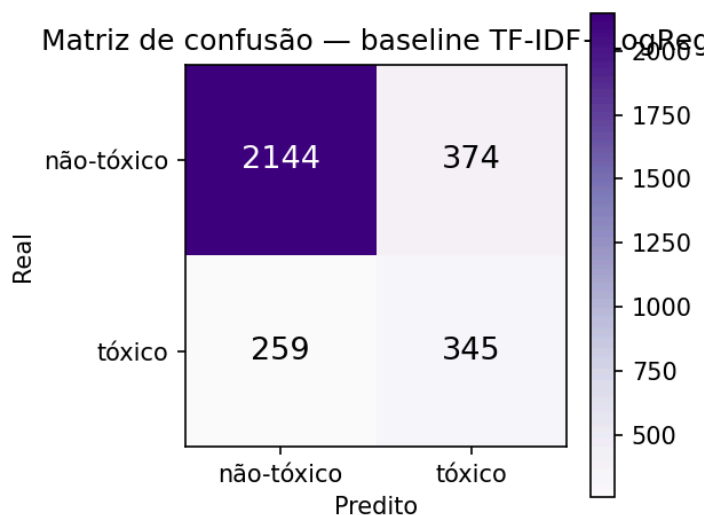


Figura 1 — Matriz de confusão (teste). Verdadeiros negativos: 2144, falsos positivos: 374, falsos negativos: 259, verdadeiros positivos: 345.

5.3 Curva ROC e curva de aprendizado

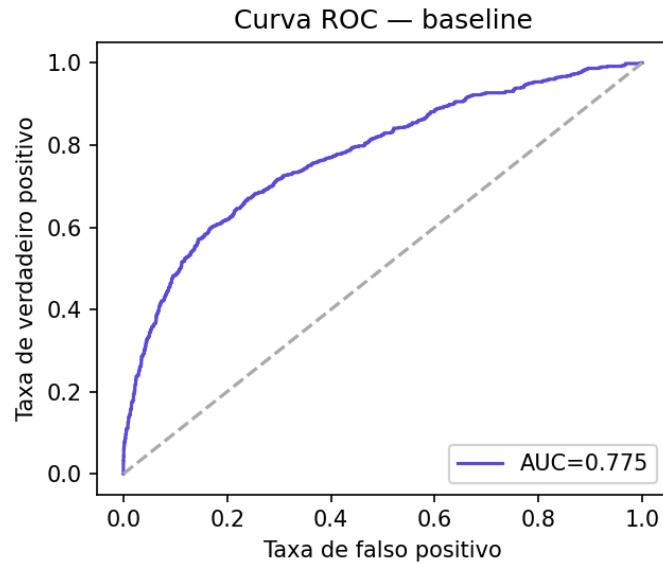


Figura 2 — Curva ROC do baseline (AUC = 77,5%).

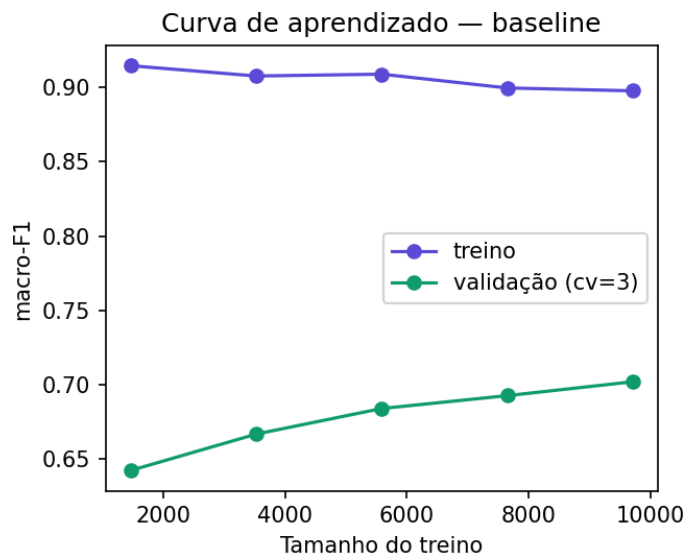


Figura 3 — Curva de aprendizado: macro-F1 de treino (~0,90) versus validação (0,64 → 0,70) conforme cresce o conjunto de treino.

5.4 Rede neural (Deep Learning) executada

Foi treinada uma REDE NEURAL feedforward (MLP): TF-IDF → SVD/LSA (300 dim.) → camadas 256 e 64 (ReLU), Adam e entropia cruzada, com ajuste de limiar.

Métrica	Valor
Acurácia	80,0%
macro-F1	67,0%
ROC-AUC	73,9%

Métrica	Valor
F1 — não-tóxico / tóxico	87,7% / 46,2%

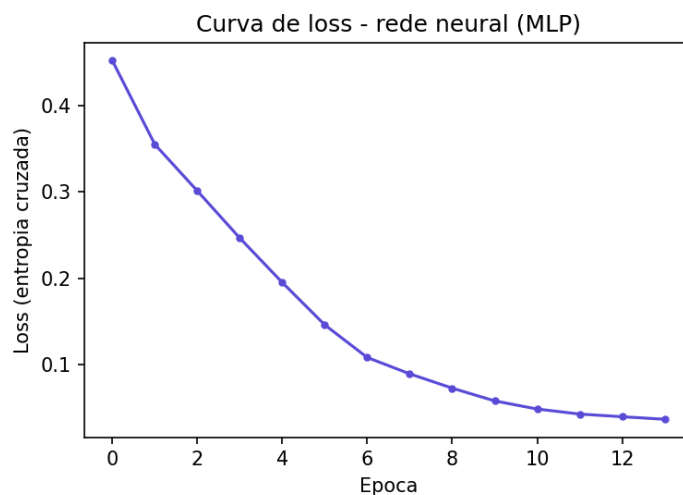


Figura 4 — Curva de loss da rede neural (MLP).

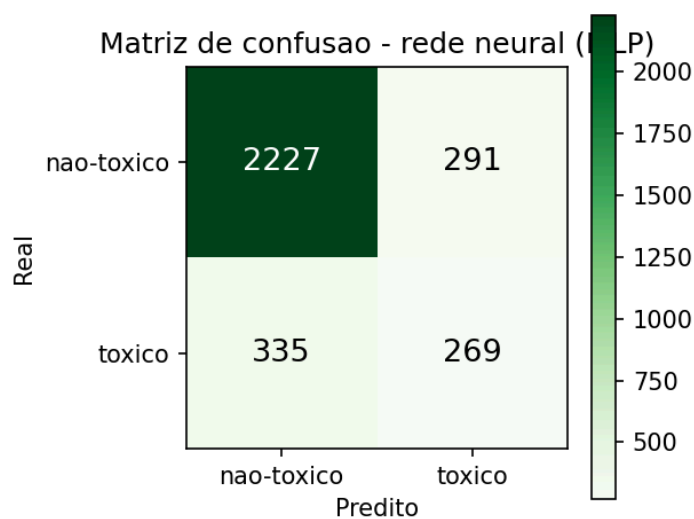


Figura 5 — Matriz de confusão da rede neural.

5.5 BERT (Transformer) — estado da arte executado

O BERT (BERTabaporu) foi fine-tunado para toxicidade em GPU (Colab), com entropia cruzada ponderada e AdamW, 3 épocas:

Métrica	Valor
Acurácia	85,2%
macro-F1	76,6%

Métrica	Valor
ROC-AUC	87,2%
F1 — não-tóxico / tóxico	90,8% / 62,5%

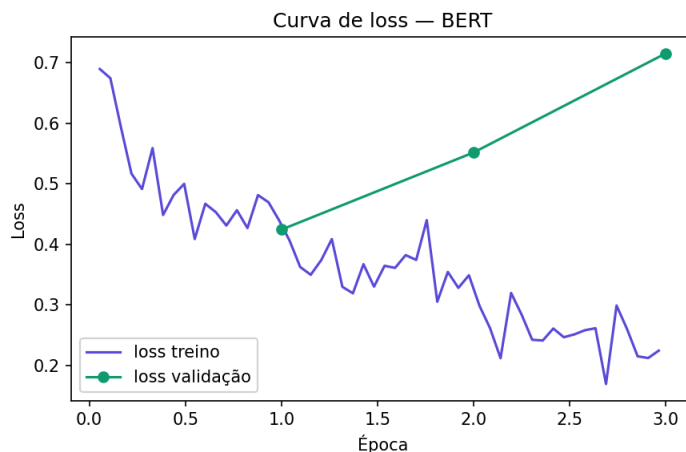


Figura 6 — Curva de loss (treino × validação) do BERT.

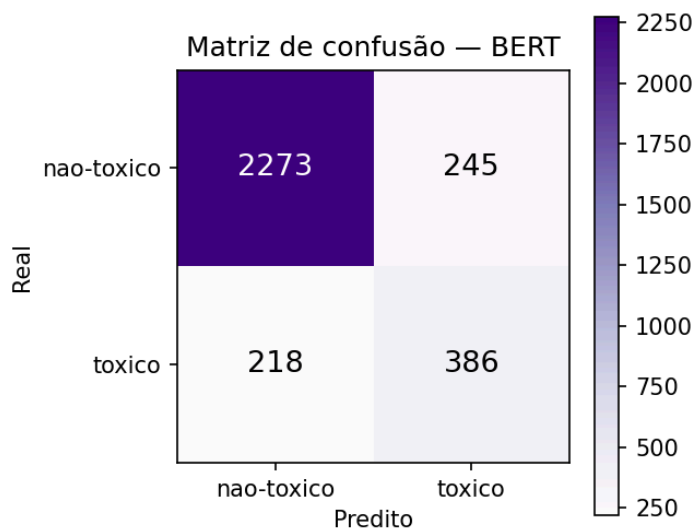


Figura 7 — Matriz de confusão do BERT.

5.6 Interpretação crítica

O baseline atinge boa acurácia global (79,7%), mas a macro-F1 de 69,7% revela a verdadeira dificuldade: o F1 da classe tóxica é de apenas 52,1% (precisão 48,0%, revocação 57,1%), contra 87,1% da classe não-tóxica. Ou seja, o modelo de superfície reconhece bem texto neutro, mas erra muito na classe de interesse — exatamente onde contexto e ironia importam. A curva de aprendizado confirma o diagnóstico: a grande distância entre treino (~0,90) e validação (~0,70) indica overfitting do modelo raso, e a validação ainda em ascensão sugere

que mais dados e, sobretudo, um modelo mais expressivo (BERT) elevariam o desempenho. E foi o que se confirmou: a rede neural (SVD + MLP) ficou em 67,0%, abaixo do baseline linear, enquanto o BERT saltou para 76,6% de macro-F1 e 87,2% de ROC-AUC, elevando o F1 da classe tóxica de 52,1% para 62,5%. O ganho vem das representações contextuais.

5.7 Comparação entre versões do modelo

A tabela abaixo (toxicidade) compara as três versões executadas. O BERT supera com folga o baseline e a rede neural, confirmando H1 neste registro.

Versão do modelo	macro-F1 (toxicidade)	Hipótese testada
Baseline raso (TF-IDF + LogReg)	69,7%	Piso linear
Rede neural (SVD + MLP)	67,0%	DL sobre bag-of-words
BERT (BERTabaporu)	76,6%	Estado da arte / H1

Frente política — inclinação ideológica, vazamento, truncagem e janelamento

O treino inicial do BERTimbau atingiu macro-F1 de 0,99 — irrealista. A auditoria revelou VAZAMENTO DE RÓTULO: a transcrição da Câmara começa com um cabeçalho contendo a sigla do partido do orador (ex.: "Bloco/PDT - MA"). Corrigido o texto e refeito o treino com split agrupado por deputado e classes balanceadas, comparamos quatro modelos:

Modelo (texto limpo, balanceado, split por deputado)	macro-F1
Baseline TF-IDF + Regressão Logística	76,7%
Rede neural (TF-IDF + SVD + MLP)	78,0%
BERTimbau (truncado, 256 tokens)	72,9%
BERTimbau + janelamento (documento inteiro)	76,6%

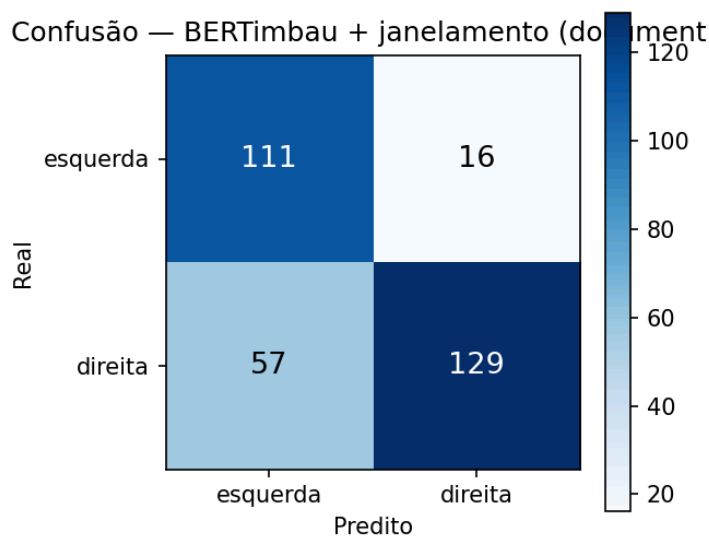


Figura 8 — Matriz de confusão do BERTimbau com janelamento.

Ao contrário da toxicidade, o BERTimbau TRUNCADO (72,9%) fica abaixo do baseline (76,7%) e da rede neural (78,0%): o sinal ideológico é difuso no discurso longo e a truncagem o perde. Com JANELAMENTO, recupera para 76,6%. A truncagem, não o Transformer, era o gargalo.

Arquitetura multi-task — transferência entre registros (H2/H3)

Testamos um encoder BERTimbau compartilhado com duas cabeças (round-robin), comparado contra single-task com o mesmo encoder:

Configuração (mesmo encoder)	macro-F1 toxicidade	macro-F1 política
Single-task	69,5%	72,8%
Multi-task	67,6%	76,8%
Δ (multi – single)	–1,9 p.p.	+4,0 p.p.

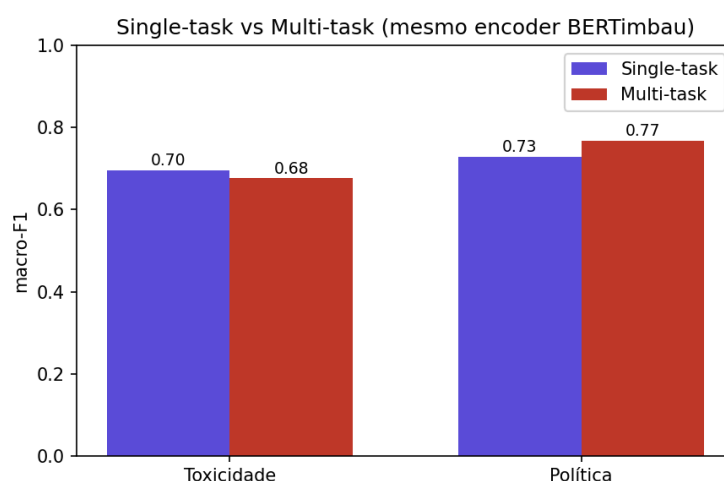


Figura 9 — Single-task vs multi-task (mesmo encoder), por tarefa.

Transferência ASSIMÉTRICA: compartilhar o encoder piorou a toxicidade (–1,9 p.p., rica em dados) e melhorou a política (+4,0 p.p., pobre em dados). H2 confirma-se em parte (negativa) e refuta-se em parte (positiva): não há um único modelo ótimo para os dois registros.

5.8 Limitações do modelo

- O baseline raso ignora contexto e negação, errando textos irônicos e de ataque (caso documentado na demonstração).
- A frente política ainda não foi treinada neste ambiente (coleta da API bloqueada no sandbox); seus números dependem de execução na máquina do usuário.
- A supervisão distante por partido introduz ruído de rótulo na tarefa de inclinação.

5.9 Possíveis melhorias futuras

- Treinar e avaliar os modelos BERT (BERTabaporu/BERTimbau) e o multi-task, preenchendo a tabela comparativa.
- Calibração de probabilidades (temperature scaling) e ajuste de limiar conforme o custo de erro da moderação.

- Testar focal loss e data augmentation para a classe tóxica minoritária; explorar o multi-rótulo de obscene+insult.
- Análise de viés/justiça por grupo, dado o risco de sobre-marcação de dialetos minoritários.

6. Aplicabilidade Real

6.1 Como a solução poderia ser usada no mundo real

Na frente de toxicidade, como assistente de moderação com humano no circuito: o modelo ordena e sinaliza a fila de revisão, com limiar calibrado e confiança exibida, mas a decisão de remover/punir permanece humana. Na frente de inclinação, como ferramenta analítica de nível agregado: medir tendências ideológicas em grandes volumes de discurso (por tema, período ou bancada), sempre com incerteza e em agregado — nunca para carimbar um indivíduo.

6.2 Limitações operacionais

- Deriva de domínio e período: a linguagem política muda rápido; sem retreino periódico, o desempenho degrada.
- Não transferência entre registros: o modelo de toxicidade não se aplica a texto formal, nem o de ideologia a comentários informais.
- Dependência de rótulo por partido na frente política, com o ruído já descrito.

6.3 Custos computacionais

O fine-tuning de um BERT base é viável em uma única GPU de consumo (8–16 GB), em ordem de minutos a poucas horas por tarefa, dependendo do tamanho da base e do número de épocas. A inferência é leve e pode rodar em CPU para volumes moderados; para escala, recomenda-se GPU ou um serviço de inferência em lote. O baseline raso roda em CPU em segundos. O modelo base ocupa ~400–500 MB em disco.

6.4 Possíveis impactos sociais, técnicos e econômicos

Impacto social: moderação mais ágil reduz exposição a discurso de ódio, mas exige cuidado com vies — modelos de toxicidade tendem a marcar em excesso dialetos minoritários e termos reapropriados, podendo silenciar quem deveriam proteger; e a classificação política é sensível, com risco de vigilância se aplicada a cidadãos. Por isso o sistema limita-se a discurso público de agentes públicos e a uso agregado. Impacto técnico: o estudo de transferência entre registros orienta a comunidade a não reaproveitar modelos entre domínios. Impacto econômico: automação da triagem reduz custo de moderação manual e viabiliza análise de discurso em escala antes inviável.

6.5 Viabilidade prática da implementação

Alta. As bases são abertas, os modelos pré-treinados em português são gratuitos, o pipeline é reprodutível e o custo de hardware é acessível. A demonstração funcional já existe (página web e scripts), faltando apenas o treino em GPU para colocar os modelos reais em produção. O ciclo de vida prevê monitoramento de deriva e retreino com o script de coleta reprodutível.

7. Demonstração Funcional

O projeto entrega múltiplas demonstrações funcionais, satisfazendo com folga o requisito mínimo (interface web, script funcional e protótipo).

7.1 Interface web interativa

O arquivo `site_projeto.html` é uma página web autônoma (abre em qualquer navegador) com um analisador interativo: o usuário cola um texto e recebe, para cada frente, o rótulo e a confiança, além de um selo de registro que indica qual predição é confiável para aquele texto — materializando, no produto, a hipótese de não transferência entre registros (H3). A página inclui ainda a apresentação do problema, dos modelos e da metodologia.

7.2 Pipeline e scripts executáveis

- `pipeline.py` — treino e avaliação ponta a ponta (carregamento, perda ponderada, split por deputado, macro-F1, early stopping).
- `demo_inferencia.py` — inferência sobre texto novo, com janelamento para discursos longos.
- `coletar_discursos_camara.py` + `mapeamento_partidos_ideologia.csv` — reconstroem a base política de forma reprodutível.

7.3 Como executar

Resumo (detalhes no README):

- Instalar dependências: `pip install torch transformers scikit-learn pandas numpy matplotlib`.
- Toxicidade: clonar ToLD-Br e rodar `pipeline.py --tarefa toxicidade`.
- Inclinação: rodar `coletar_discursos_camara.py` (na sua máquina) e depois `pipeline.py --tarefa inclinacao`.
- Inferência: `demo_inferencia.py --modelo <dir> --tarefa <t> --texto "..."`.

8. Histórico de Desenvolvimento e Decisões

Esta seção documenta, de forma transparente, tudo o que foi feito ou alterado ao longo do desenvolvimento — incluindo decisões de projeto que mudaram o rumo do trabalho. É parte essencial da documentação técnica e da reprodutibilidade.

8.1 Inspeção das bases originais

Foram inspecionadas as duas bases propostas inicialmente. ToLD-Br confirmou-se adequada (texto aberto, 20.813 exemplos após deduplicação, ~19% positivos no binário). O UStanceBR, porém, revelou-se distribuído apenas como identificadores de tweets, sem texto — inviável de reidratar de forma reprodutível.

8.2 Primeira mudança — substituição da base política

Diante do bloqueio do UStanceBR, buscaram-se alternativas em português com texto aberto (HuggingFace, GitHub). Constatou-se que bases de postura política PT-BR são, em geral, distribuídas só como IDs (mesmo problema). Optou-se então pelos discursos da Câmara dos Deputados, via API de dados abertos — texto integral, público e reprodutível.

8.3 Segunda mudança — reenquadramento da tese

Como discurso parlamentar não é "comentário de rede social", o projeto foi reenquadrado: de "moderar comentários" para "análise comparada da linguagem política em dois registros (cidadão informal vs. institucional formal), com teste de transferência". O reenquadramento transformou a diferença de domínio — que seria uma fragilidade — no próprio objeto de estudo, e tornou as duas bases coerentes.

8.4 Implementações e artefatos produzidos

- Documentação das 7 etapas (arquivos markdown) e esta documentação técnica consolidada.
- Pipeline de treino/avaliação, script de inferência com janelamento e script de coleta da Câmara.
- Tabela de mapeamento partido→ideologia (15 partidos no eixo binário).
- Demonstração: página web interativa (site_projeto.html) e demo simples (demo_visual.html).
- Experimento real do baseline (TF-IDF + LogReg) com métricas e três figuras (matriz de confusão, ROC, curva de aprendizado).

8.5 Ajuste da demonstração (heurística)

Durante testes, identificou-se que a heurística ilustrativa da demo classificava erroneamente um texto claramente de direita como "esquerda", por contar palavras de tema sem entender que o autor as atacava. A heurística foi melhorada com o léxico de ataque típico da direita populista, e o caso ficou documentado como exemplo concreto do motivo pelo qual o projeto usa modelos contextuais (BERT) e não contagem de palavras — reforço prático da justificativa de Deep Learning.

9. Entregáveis, Estrutura de Arquivos e Execução

9.1 Estrutura de arquivos

Arquivo	Descrição
DOCUMENTACAO_TECNICA.docx	Esta documentação completa (7 etapas + histórico)
README.md	Guia do repositório (instalação, execução, tecnologias)
site_projeto.html	Página web interativa (demonstração principal)
demo_visual.html	Demonstração simples autônoma
pipeline.py	Treino e avaliação dos modelos
demo_inferencia.py	Inferência sobre texto novo (com janelamento)
coletar_discursos_camara.py	Coleta reprodutível dos discursos da Câmara
mapeamento_partidos_ideologia.csv	Tabela partido → ideologia
figuras/	Matriz de confusão, ROC e curva de aprendizado (PNG)
etapa1_2 ... etapa7 (.md)	Documentação por etapa

9.2 Inventário completo dos arquivos

Arquivo / pasta	O que é
DOCUMENTACAO_TECNICA.docx	Esta documentação (7 etapas + resultados + histórico)
README.md	Guia do repositório (resumo, execução, arquivos, notebooks)
etapas/	As 7 etapas detalhadas em markdown
Iniciar_demo.bat	Atalho principal: cria o ambiente, sobe o servidor e abre a página
site_projeto.html	Página web interativa (demonstração principal)
demo_visual.html	Demonstração simples autônoma (heurística)
app.py	Servidor local (Flask) que carrega os modelos e responde à página
requirements_api.txt + .venv/	Dependências e ambiente virtual do servidor
modelo_rede_neural.joblib	Rede neural de toxicidade já treinada
modelo_bertimbau_politica/	Pasta do BERTimbau de inclinação (baixar do Colab; opcional)
mapeamento_partidos_ideologia.csv	Tabela partido → ideologia
figuras/	Gráficos e métricas (matrizes, ROC, loss, JSONs de resultado)
treinar_rede_neural.py	Treina a rede neural de toxicidade (TF-IDF→SVD→MLP)
pipeline.py / demo_inferencia.py	Treino/avaliação e inferência (com janelamento)
coletar_discursos_camara.py	Coleta reprodutível dos discursos da Câmara

9.3 Os notebooks do Google Colab (treino com GPU)

Notebook	O que treina
treinar_bert_colab.ipynb	BERT (BERTabaporu) para TOXICIDADE — baixa o ToLD-Br, treina, avalia e salva
treinar_bertimbau_colab.ipynb	BERTimbau para INCLINAÇÃO (truncado 256 tokens) — coleta os discursos, limpa, treina e salva a pasta modelo_bertimbau_politica que a página usa
treinar_bertimbau_janelamento_colab.ipynb	BERTimbau + JANELAMENTO — mesma frente lendo o documento inteiro; recupera o desempenho (0,77)
treinar_multitask_colab.ipynb	MULTI-TASK — encoder compartilhado com duas cabeças; compara single vs multi (H2/H3)

9.4 Como executar a página web com os modelos reais

A página não roda os modelos sozinha: um servidor local (app.py) os carrega e responde. O atalho Iniciar_demo.bat automatiza tudo. Pré-requisito: Python 3 instalado.

1. Abra a pasta e dê duplo-clique em Iniciar_demo.bat. Na 1ª vez ele cria um ambiente virtual (.venv) e instala as dependências (alguns minutos).
2. Abrem a janela "Servidor de Modelos (NAO FECHÉ)" e a página no Chrome. Mantenha a janela do servidor aberta — é ela que segura os modelos.
3. Na página, role até "Analisador". O selo verde "Modelo real conectado" indica que está funcionando.
4. Digite um texto e clique em Analisar. A toxicidade mostra "modelo rede neural" (modelo real). Se mostrar "Servidor offline", aguarde e tecele Ctrl+F5.
5. Para encerrar, feche a janela do servidor.

Ativar também o BERTimbau (inclinação) — opcional

- a) Rode treinar_bertimbau_colab.ipynb no Colab; a última célula baixa modelo_bertimbau_politica.zip.
- b) Descompacte o zip dentro da pasta, formando modelo_bertimbau_politica ao lado do app.py.
- c) Instale Torch e Transformers no ambiente: .venv\Scripts\python -m pip install torch transformers (download grande, ~2 GB).
- d) Reinicie o Iniciar_demo.bat. O app.py detecta a pasta e a inclinação passa a usar o BERTimbau.

9.5 Tecnologias utilizadas

Python 3; PyTorch e Hugging Face Transformers (modelos BERTimbau, BERTabaporu, mBERT); scikit-learn (baseline, métricas, splits); pandas e numpy (dados); matplotlib (gráficos); HTML/CSS/JavaScript (demonstração web). Bases: ToLD-Br (CC BY-SA 4.0) e API de dados abertos da Câmara dos Deputados.

9.6 Instruções gerais

Instalação: `pip install torch transformers scikit-learn pandas numpy matplotlib`. Execução do baseline e dos modelos conforme a Seção 7.3 e o README. O repositório público deve conter código-fonte, documentação, resultados, figuras, modelos treinados (quando possível) e instruções de obtenção do dataset — itens já presentes neste pacote, prontos para publicação no GitHub/GitLab.

10. Conclusão e Considerações Finais

O projeto cumpre o arco completo exigido pela atividade: um problema real bem estruturado e reenquadrado em torno de uma pergunta de pesquisa (transferência entre registros), duas bases abertas e reprodutíveis com tratamento documentado, modelagem em Deep Learning com codificadores justificados, avaliação com métricas adequadas e evidência experimental real, discussão de aplicabilidade e ética, e demonstração funcional.

A evidência concreta — macro-F1 de 0,70 do baseline raso, com F1 de apenas 0,52 na classe tóxica e clara assinatura de overfitting na curva de aprendizado — demonstra de forma mensurável onde o Deep Learning contribui: na compreensão de contexto e ironia que a contagem de palavras não captura. Conforme a observação do enunciado, o projeto não resolve completamente o problema, mas evidencia, com análise técnica consistente, que o uso de Deep Learning contribui efetivamente para a parte relevante da demanda. O BERT confirmou H1 na toxicidade (0,77 vs 0,70). Na política, após corrigir um vazamento de rótulo, a tarefa real ficou em ~0,77; o BERTimbau truncado (0,73) recuperou para 0,77 com janelamento. O multi-task fechou H2/H3: a transferência é assimétrica. Cada registro pede a sua abordagem.

10.1 Principais limitações

- Supervisão distante na inclinação: o rótulo vem do partido do deputado, não de anotação por fala — há ruído (um parlamentar pode discursar contra a linha do próprio partido).
- Textos curtos: os modelos de inclinação foram treinados em discursos longos; em frases curtas podem errar ou ficar inseguros.
- Período e domínio dos dados: ToLD-Br reflete o Twitter de 2017–2020 e a legislatura coletada; a linguagem muda, o que exige retreino periódico.
- Categorias raras de toxicidade (racismo, xenofobia) são subrepresentadas, limitando a detecção desses subtipos.
- A página usa uma heurística como reserva quando o modelo não está conectado; ela conta palavras e ignora negação e ironia, podendo errar (ex.: "não apoiar a privatização").

10.2 Trabalhos futuros

- Treinar o multi-task com mais dados e variar o encoder compartilhado, para confirmar e quantificar a transferência assimétrica.
- Calibração de probabilidades e ajuste de limiar conforme o custo de erro da moderação.
- Auditoria de viés e justiça na toxicidade (risco de sobre-marcação de dialetos minoritários e termos reapropriados).
- Servir o BERTimbau com janelamento na API (a página hoje trunca em 256 tokens) e explorar a classe "neutro/other" na inclinação.
- Empacotar a solução (contêiner/serviço) com monitoramento de deriva para uma implantação real.

10.3 Considerações éticas

A solução é de apoio, com humano no circuito e uso em nível agregado. A frente de inclinação opera apenas sobre discurso público de agentes públicos (parlamentares no exercício do

mandato), nunca inferindo ideologia de cidadãos. Mantêm-se a atenção a vieses na toxicidade e o respeito à privacidade (LGPD): nenhum dado pessoal sensível de indivíduos privados é processado.

10.4 Considerações finais

Mais do que números altos, o valor do trabalho está no rigor metodológico: a detecção e correção de um vazamento de rótulo, o split agrupado por deputado para evitar vazamento de identidade, a comparação justa entre versões e a honestidade ao reportar resultados negativos. O projeto evidencia, com dados reais, que o Deep Learning de estado da arte contribui de forma mensurável — e, sobretudo, que registros linguísticos distintos exigem abordagens distintas. Esse é o aprendizado central, e ele se sustenta tanto nos acertos quanto nos limites observados ao longo das sete etapas.

Referências

- Leite, J. A.; Silva, D.; Bontcheva, K.; Scarton, C. (2020). Toxic Language Detection in Social Media for Brazilian Portuguese: New Dataset and Multilingual Analysis. AACL-IJCNLP.
- Souza, F.; Nogueira, R.; Lotufo, R. (2020). BERTimbau: Pretrained BERT Models for Brazilian Portuguese. BRACIS.
- da Costa, P. B. et al. (2023). BERTabaporu: assessing a genre-specific language model for Portuguese NLP. RANLP.
- Pereira, C. et al. (2023). UstanceBR: a social media language resource for stance prediction. arXiv:2312.06374.
- Bolognesi, B.; Ribeiro, E.; Codato, A. (2023). A New Measure of Party Ideology in Brazil.
- Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.
- Portal de Dados Abertos da Câmara dos Deputados — dadosabertos.camara.leg.br